

Team Information

FarmAlot team #7

- Mansi Bhardwaj → 1866168
- Daniel Cheishvili → 1852889
- Bhavik Patel → 1825364

Project Description

The primary purpose of this project is to create an application that communicates with the container using azure. The container will send data to azure and the app will display data to the user. The app can set thresholds as well as control any hardware needed. For this to be accomplished, we used Xamarin Forms for the application, Azure to store the container's data and control the hardware, and the Raspberry Pi, which has a Location system, a Plant system, and a Security system.

Mobile App

- App Purpose

The purpose of the app is to display any data set by the container. The app receives the data from azure and displays the data to the user. The app also could control any actuators that the container has like a fan, light buzzer, and door lock.

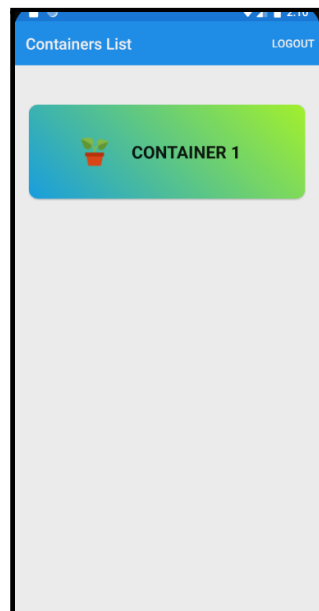
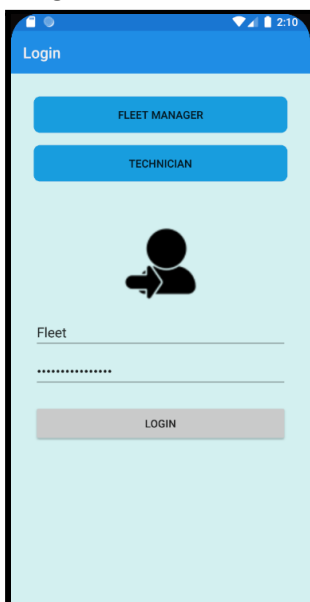
- App Functionality

The functionality of the app is quite simple. It is designed to give users remote access to all the systems to control and view data.

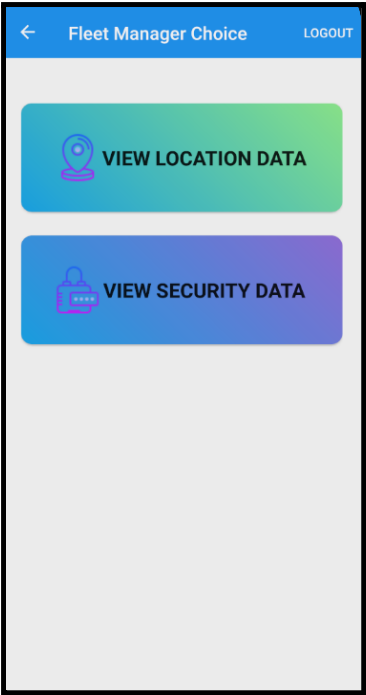
The app start where an employee of the container must log in according to their title and department. Once authenticated, they will be presented with data according to their role. The Fleet Manager will be able to view all data regarding the Location and Security of the container as well as control any actuators from within the app. The Fleet Manager will also be able to set thresholds on the app in order to determine when the actuators need to be set off. The Technician has their own view where they can control different hardware system for the plant located in the container.

- App Snapshots

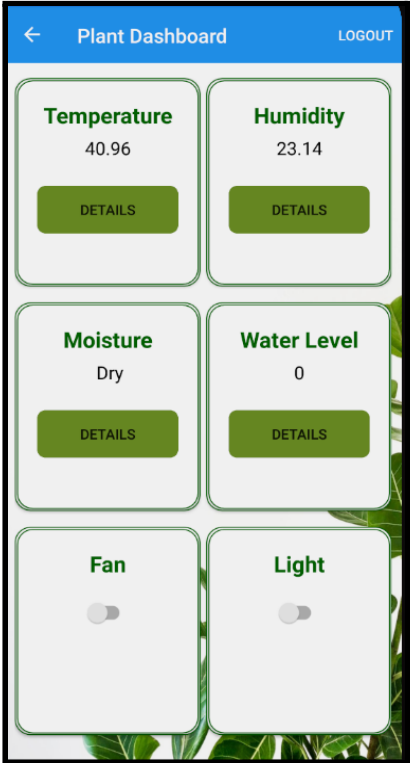
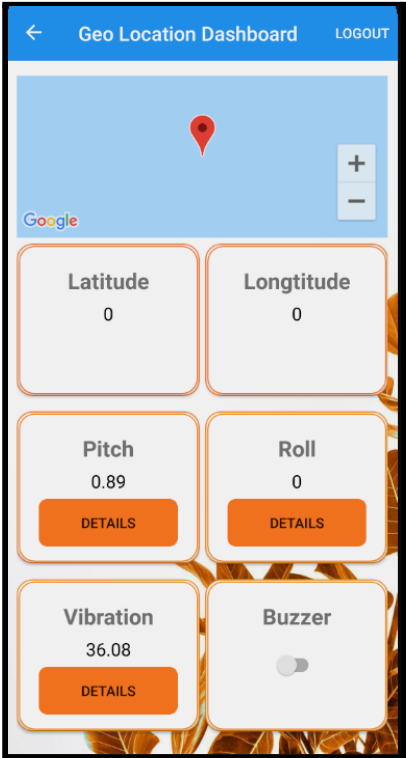
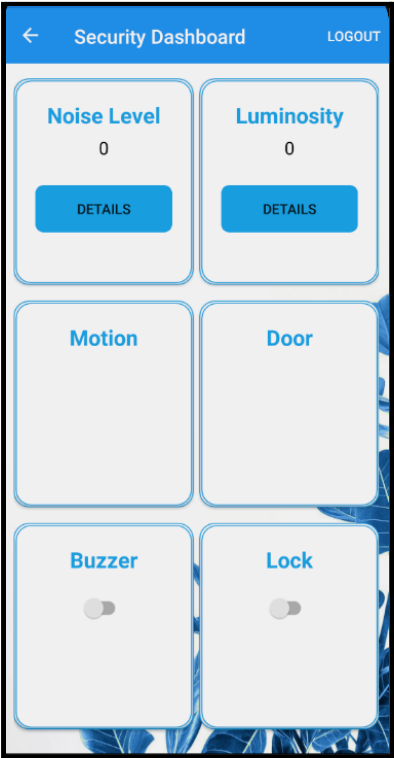
Login → Container



Fleet view



Security Data View and Geo Data View and tech View



- Future Work

We wanted to implement proper user Authentication like done in class with the OAuth packages. However, due to time restraints we had to find a quicker and simpler solution in order to do so. We got the user input from the user entry and checked in the back end if it matches the credentials.

Another feature we wanted to include in our App was to be able to share the data collected from Azure in a text via email. Again because of the lack of time, we thought this was a feature we can implement in future if more time given.

After finalizing our application, we believe some additional features that would be beneficial to our app would be storage of the data. In the future we look forward on adding data for multiple containers.

Contribution

Team members' name	Responsibility
Daniel Cheishvili	● Implemented user authentication ● Created all models, view models, repositories, and views related to the Geo Location subsystem ● Assisted in writing code to receive data from azure, and control actuators. ● Presented certain data in graphs. ● Worked on setting thresholds from the app
Mansi Bhardwaj	● Created all models, view models, repositories, and views related to the Plant subsystem ● Cleaned up and designed UI ● XML Documentation ● Updating Jira board
Bhavik Patel	● Implemented Logout Operation ● Created all models, view models, repositories, and views related to the Security subsystem ● Assisted in writing code to receive data from azure, and control actuators. ● Worked on the graph functionality ● Fixed many bugs

Receiving data from azure

To connect and receive messages from azure you need to do the following:

In the Shared project there is a folder called Azure and in that folder there is a class called AzureConfig.cs. You will need to modify the following strings because you will not have access to our IoT Hub.

EVENTS_HUBS_CONNECITON_STRING: you could get this string by going to your created iot hub in azure -> Built-in endpoints and under Event Hub compatible endpoint copy and paste the string.

CONSUMER_GROUP: it could stay as \$Default because that is the default consumer group unless you want to create a new one then you could do that and you can find this under the same tab as the previous string under Consumer group.

STORAGE_CONNECTION_STRING: go to your created storage, under Access Keys press show keys and copy the first connection string

BLOB_CONTAINER_NAME: go to your created storage, under Containers copy the name of the created container

IOT_HUB_CONNECTION_STRING: go to your created IOT Hub, under Shared access policies chose a policy name, preferably a policy where you have read and write permissions, click on it and copy the primary connection string

DEVICE_ID: go to your created IOT Hub under Devices, copy the name of your created device
Once you have all of this set up you should start receiving message from azure.

Bonus

User Authentication

For the user authentication we used a simple authentication where we bind the password and username, get the user input and check if our view model if the entered credentials match our credentials.

We ended up using this approach because due to lack of time we couldn't implement a fully functioning authentication mechanism as we did in class therefore we ended up using something simple yet effective in our project.

Those are generic users we created that could be used by any fleet manager and technician. We didn't see the point of creating a user for each role as they have the same role in the container. It is also easier to control when you have one user for the same role for all employees. Because for now there are only two roles, generic users make sense, but as it grows of course you would have to make individual users and assign them to different user groups depending on what permissions they might have or not.

Username: Tech

Password: techPassword123

Or

Username: Fleet

Password: fleetPassword123

OOP Design

We used 3 models in our development. Security, Geo-Location, and Plant Models. Each model accomplished the same thing but for different subsystems, it kept the data sent from azure and stored it. It then communicated with the view model in which the view model then displays the data to the users in the view.

UML Diagram

https://lucid.app/lucidchart/b70904f5-c1ac-4893-943c-d0502a727b77/edit?invitationId=inv_83e57e1c-4de6-4b39-be75-5648c9434edd

Connected Objects

IoT System

Dummy payload

Line 47 in Security.py could be changed to: msg_txt_formatted =

MSG.TXT.format(noise=random.randint(0,600), luminosity=random.randint(0,15000),
motion=true or false, door=lock or unlock)

Line 111 in PlantTelemetry.py could be changed to: msg_txt_formatted =

MSG.TXT.format(temperature=random.randint(0,40), humidity=random.randint(0,100),
moisture=dry wet or moist, water=0,1000)

Line 166 in GeoLocationSubsystems.py could be changed to: msg_txt_formatted =

MSG.TXT.format(latitude=random.randint(0,70), longitude=random.randint(-70,0),
pitch=random.randint(-90,90), roll=random.randint(-90,90),
vibration=random.randint(0,1000))

Sensors

- Plant
 - Water Level → A0
 - Temp_Humi → D26
 - Moisture Sensor → A2
 - Fan → D22 (Relay)
 - Chainable Light → plugged in the back in the ports of number 14(TX),15(RX) GND and 3v3
- Security
 - Noise sensor → A4
 - Motion Sensor → PWM
 - Door → D5
 - Door Lock → D24
- Geo Location:
 - GPS → UART port

Cloud-to-Devices

1. Choice of communication
 - a. As a team, we decided to use Device Twins to control actuators
2. Explanation of method
 - a. The reason why we decided to use Device Twin was because we were already receiving device twin updates for controlling the interval so it was easier to just modify the method that is used to control the interval to as well control the actuators. Also for the App Dev milestone we are required to use Device Twins in order to get the interval. Therefore, using one method instead of multiples makes the project simpler and more time efficient
3. Keys and values used
 - a. "fan" : "on/off"
 - b. "buzzer" : "on/off"
 - c. "door_locked" : "unlock/lock"
 - d. "light" : "on/off"
 - e. "Noise_threshold" : any double value
 - f. "Lumi_threshold" : any double value
 - g. "Vib_threshold" : any double value
 - h. "Temp_threshold" : any double value
4. Example for how one of the actuators can be controlled from the device interface of the Azure Portal. To control the actuators just change the value of the properties below mentioned answer 3 and run the script.

```
19      "region": "US",
20      "plant": "Redmond43"
21    },
22  },
23  "properties": {
24    "desired": {
25      "power_level": 1,
26      "smart": 99,
27      "interval": "hello",
28      "telemetryInterval": 2,
29      "buzzer": "off",
30      "door_locked": "unlock",
31      "fan": "off",
32      "light": "off",
33    },
34    "reported": {
35      "$lastUpdated": "2022-05-14T23:23:46.6283149Z",
36      "$lastUpdatedVersion": 119,
37      "power_level": {
38        "$lastUpdated": "2022-05-14T23:23:46.6283149Z",
39        "$lastUpdatedVersion": 119
40      }
41    }
42  }
```

Contribution

- Mansi:
 - Implemented a functioning Plant System
 - All actuators of the plant system could be controlled using Device Twins
 - All data that could be read is sent to azure
 - Assisted in combining all subsystems into one class and sending in one payload instead of 3
- Daniel:
 - Implemented a functioning Geo Location System
 - All actuators of the geo location could be controlled using Device Twins
 - All data that could be read is sent to azure
 - Assisted in combining all subsystems into one class and sending in one payload instead of 3
- Bhavik:
 - Implemented a functioning Security System
 - All actuators of the security subsystem can be controlled using Device Twins
 - All readable data is sent to azure

- Helped combine all 3 payloads into one.
- All: Worked on AllSubsystems.py together, connected all the components to one raspberry pi.

Getting Started (Bonus)

Welcome! We are thrilled that you want to configure our own raspberry Pi to a Farm container. Our container has three different subsystem: Plant, Security and Geo Location

This section contains step-by-step instructions on how to get started with this Farm container. In this tutorial, you'll learn how to:

- Connect all Hardware to the base hat
- Controlling and reading Data from sensors
- Connecting to an Azure portal Account
- Send data to Azure
- Receive Data from Azure

Hardware

You will need:

Plant Subsystem

- [Water Level Sensor](#)
- [Soil Moisture Sensor](#)
- [Chainable RGB Led](#)
- [Cooling Fan](#)
- [AHT20 Temp & Humidity Sensor](#)

Geo Location Subsystem

- [GPS \(Air530\)](#)
- [Accelerometer](#)
- [reTerminal's built-in buzzer](#)

Security Subsystem

- [PIR Motion Sensor](#)
- [Magnetic door sensor reed switch](#)
- [MG90S 180° Micro Servo](#)
- [Sound Sensor/ Noise Detector](#)

Connection to Base Hat

Next you will have to connect all the hardware to the raspberry pi through the [Base Hat](#).

Connect the :

- Water Level → A0 port

- Moisture Sensor → A2 port
- Fan → D22 (Relay)
 - Fan Pins → The 5V pin is available on the underside of the Base Hat, where the GPIO pins are located. The 5V pin sits opposite to physical pin #1. Pin #1 is labeled on the top-side of the Base Hat. Attach the remaining cable of the relay to the GND pin of the Base Hat. Third from the 5V pin.
- Chainable Light → RX TX from the underside of the Base Hat
- Temperature Sensor → D26
- Noise sensor → A4 port
- Motion Sensor → PWM port
 - [PIR Motion Sensor](#)
 - In the back of the sensor, make sure to turn the *OnTime* screw all the way to the right and the *sens* screw all the way to the left.
- Door → D5 port
- Door Lock → D24 port
 - Use this [tutorial](#) to see how to connect it since the micro servo only has 3 female pins so connect them using the basehat to male pins cable for easier usability.
- GPS → UART port

Fetching Data From Sensor

Before moving on to any other step, you must make sure all devices connected to the raspberry pi are working accordingly.

For the plant subsystem you will need to follow these libraries.

- Water Level → [ADC library for Grove Base Hat](#)
- Moisture Sensor → [ADC library for Grove Base Hat](#)
- Fan → [LED](#)
- Chainable Light → [chainable_rgb_direct_example.py](#)
- Temperature Sensor → [Grove_temperature_humidity_aht20.py](#)
 - The temperature sensor is the same address of the reTerminal's built-in buzzer, LEDs and buttons. In order to avoid this problem you will create a new I2C bus for the AHT20. To connect the sensor to the Base Hat you will create a new bus on GPIOs 26 and 27 by following this [simple tutorial](#).
 - Navigate to the i2c.py file and follow this: <https://github.com/Seeed-Studio/grove.py/issues/12>
- Noise sensor → [Sound Sensor/ Noise Detector](#)
- Motion Sensor → [Motion Sensor Library](#)
- Door → Same principal as the Button class from [Gpiozero](#)
- Door Lock → [Micro servo example](#) and Micro servo
 - Code example from the first link, use AngularServo example to start off and change the values to your required values.
- GPS → Make sure to carefully follow the [GPS data](#) tutorial

Connection To Azure Portal Account

For the container to send data and read data from a cloud device, you will use the Azure Portal to create an IoT Hub and a device. Then with the use of this device we will be able to use the Azure CLI to simulate a device, send device telemetry to the hub, and send a cloud-to-device message.

Follow this [Quickstart: Send telemetry from a device to an IoT hub and monitor it with the Azure CLI](#).

1. **Create an IoT Hub** using the Azure Portal (web interface).
 - a. Follow this [guide for details](#).
2. **Register a new device** using the Azure Portal.
 - a. Follow this [guide for details](#).

Once your device is ready to go, you are now ready to send the information collected from the sensors to Azure. You may wish to connect your Azure account to [VS Code](#).

You will need to gather all the connection strings from your IOT Hub.

- IOTHUB_DEVICE_CONNECTION_STRING
- IOTHUB_CONNECTION_STRING
- IOTHUB_DEVICE_ID

Sending Data

You will need to create a client and send a payload message that will include information from the sensor.

Refer to this github repository for a better understanding on how to connect and send data

- [Azure Send Data Repo](#)

Verify that you are receiving the messages via the vs code extension or the Portal

Receiving Data

Now that you can successfully send data to the IOT hub, you will now control your sensors through the portal. There are several different ways of doing so:

- Direct Method
- Device Twin

We will be using Device twin to receive instructions to control our sensors. Device twin are JSON documents that contains the device state including meta data, configuration and conditions.

In order to control the sensors, the part from the JSON document we need the most is the desired properties and reported properties. The desired properties and reported properties allows us to synchronize device configuration and conditions.

When sending data from the Raspberry Pi, you can set the desired properties and the device app can read them. The device app can set reported properties, and in return the back end can read and query them.

Follow this guide on how to handle receiving data and manipulating it.

- [Get Started w/ Device Twins](#)
- Follow the example ReportConnectivity.py, the twin path method

Make sure to set the same words you receiving as the key from the twin patch.

Congratulations! You now have a working Container.

Following Part of the tutorial from Maurio Buschinelli:

Defining a new I2C Bus on the Pi

1. List the default I2C buses. Once created, your new bus will appear here.

```
pi@raspberrypi:~$ i2cdetect -l
```

i2c-3	i2c	bcm2835 (i2c@7e205600)	I2C adapter
i2c-1	i2c	bcm2835 (i2c@7e804000)	I2C adapter
i2c-21	i2c	Broadcom STB :	I2C adapter
i2c-20	i2c	Broadcom STB :	I2C adapter

2. List all built-in devices connected to I2C-1 (assuming there are no external devices connected). This is the default I2C bus which is connected to GPIOs 2 and 3.

```
pi@raspberrypi:~$ i2cdetect -y 1
```

```
      0 1 2 3 4 5 6 7 8 9 a b c d e f
00:      -- 04 - - - - - - - - - -
```

```

10: ----- UU -----
20: ----- UU -----
30: ----- UU -----
40: ----- UU -----
50: -----
60: -----
70: -----

```

Confirm that the address 0x38 is already being used. This is the I/O Expander.

3. Make a copy of the file `/boot/config.txt` and name it `/boot/config_backup.txt`. Hopefully you won't need this.
4. Add the line below at the end of your `/boot/config.txt` file.

```
dtoverlay=i2c-gpio,bus=4,i2c_gpio_delay_us=1,i2c_gpio_sda=27,i2c_gpio_scl=26
```

5. List all available I2C buses. **i2c-4** is your newly created I2C bus #4.

```
pi@raspberrypi:~ $ i2cdetect -l
```

i2c-3	i2c	bcm2835 (i2c@7e205600)	I2C adapter
i2c-1	i2c	bcm2835 (i2c@7e804000)	I2C adapter
i2c-21	i2c	Broadcom STB :	I2C adapter
i2c-4	i2c	400000002.i2c	I2C adapter
i2c-20	i2c	Broadcom STB :	I2C adapter

6. Connect your AHT20 sensor to the Base Hat on the correct GPIO

```
pi@raspberrypi:~ $ i2cdetect -y 4
```

```

      0 1 2 3 4 5 6 7 8 9 a b c d e f
00:  -----
10:  -----
20:  -----

```

30: -----38-----
40: -----
50: -----
60: -----
70: -----